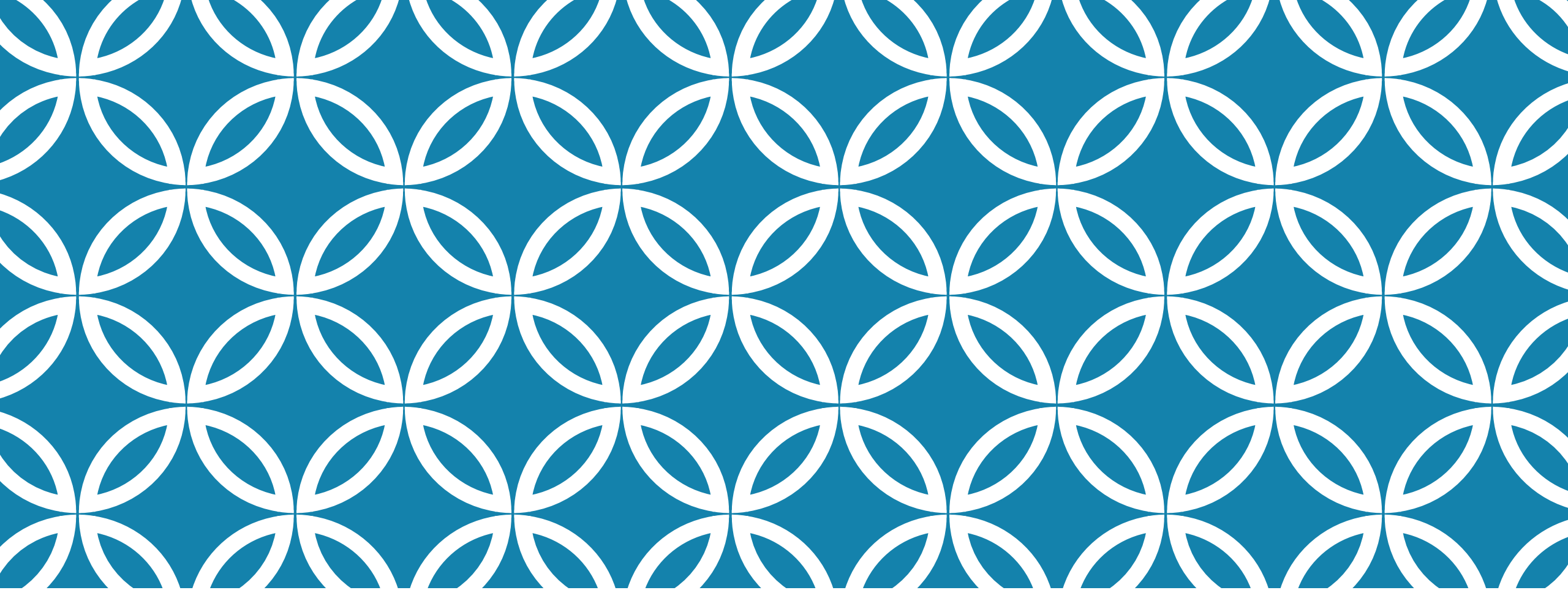
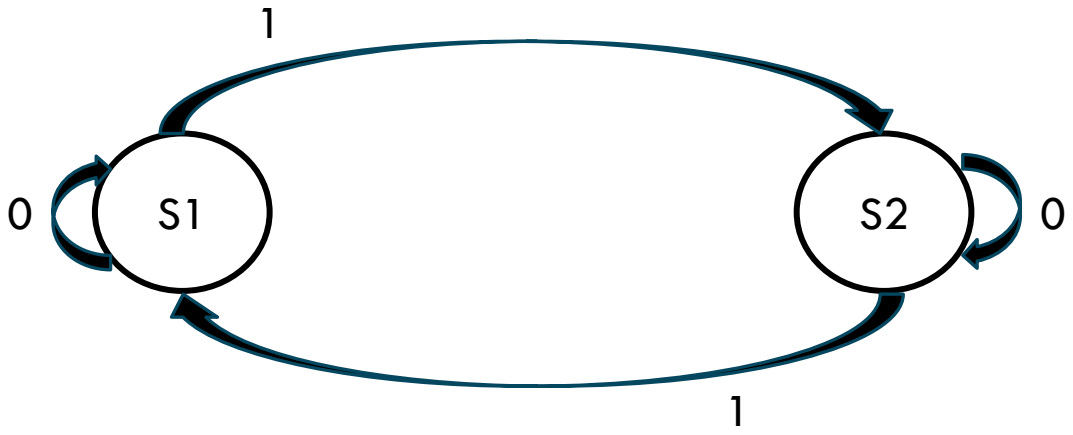


ENG GEN 225 DIGITAL CIRCUITS LAB 4

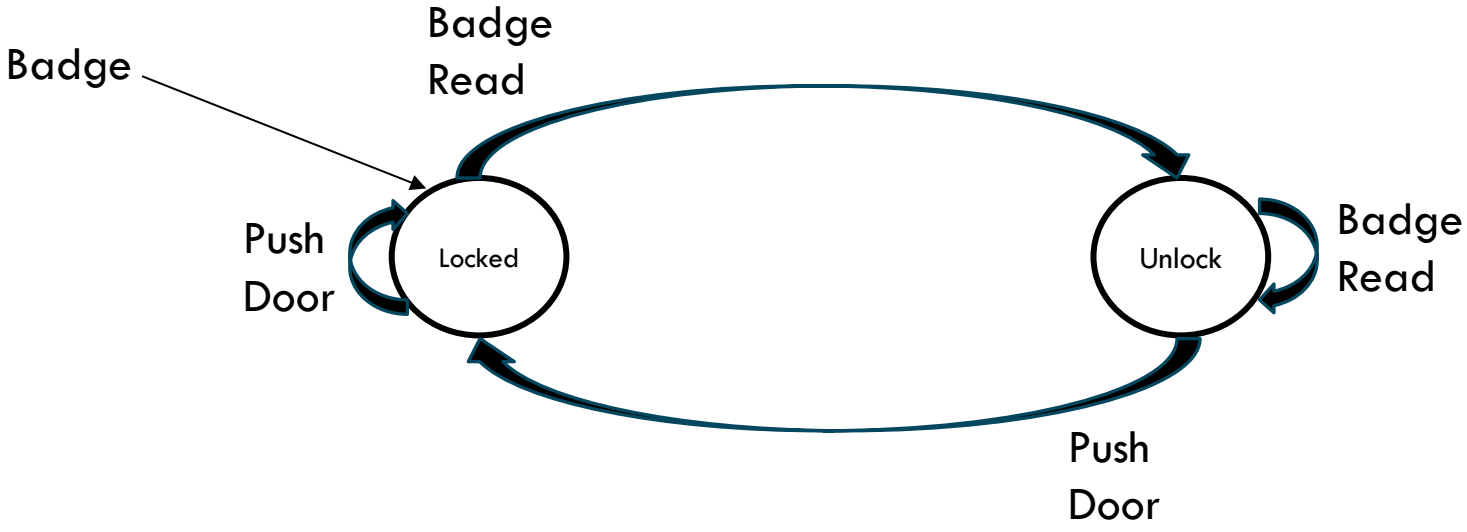
STATE MACHINE

- A “machine” that can be in exactly one finite number of states at any given time
- Transitions from one state to another in response to inputs



BADGE DOOR OPENER

- Example: You need a badge to enter your work. If you push the door without tapping badge to reader the door remains locked (S1). If you tap your badge on the reader the door will unlock (S2). If you keep on tapping badge while door is unlock it will remain unlocked until you push the door open, pass through. Once the door closes behind you it locks or goes back to S1.



NEW SYNTAX

- localparam CONSTANT_NAME //a parameter local to module that cannot be overwritten by external parameter redefinition

- assert (*statement*) //used in simulation to check a statement. Verilog does not recognize, only SystemVerilog

- always @(sensitivity)

```
begin
  if (sensitivity)
   statement
  else
  begin
      case (variable)  //variable usually a register
      case1_name:
                if (sensitivity) //sensitivity to an input
                   statement  //variable <= state;
      case2_name:
                if (sensitivity)
                statement
     endcase
  end
end
```

- case (*variable*)

```
  case1: statement  //case can be a name or a value (e.g. 3'b000)
  case2: statement
  case3: statement
  case4: statement
endcase
```

BLOCKING VS NON-BLOCKING ASSIGNMENTS

= *Blocking Assignment: Evaluation and assignment are immediate*

Example:

```
a=0, b=0;
a=1;        //  immediately assigns new value to a (a=1)
b=a;        //  a updated to1 so now b =1
a=1, b=1
```

<= *Nonblocking Assignment: All assignments defer until all evaluations have completed*

Example:

```
a=0, b=0;
a<=1;       //  Evaluates a=1 but defer assignment
b=a;        //   Evaluates b=a and immediately assigns value
a=1, b=0
```

BLOCKING VS NON-BLOCKING ASSIGNMENTS

- As projects get more and more complex we need to establish guidelines for when to use blocking or nonblocking assignments

- Best practice is to use non-blocking assignments for sequential (clock sensitivity) always blocks

Ex.

```
Always @(posedge i_Clk)
begin
 x <= a|b;
 y <= a^b;
 z <= a&b;
end
```

BLOCKING VS NON-BLOCKING ASSIGNMENTS

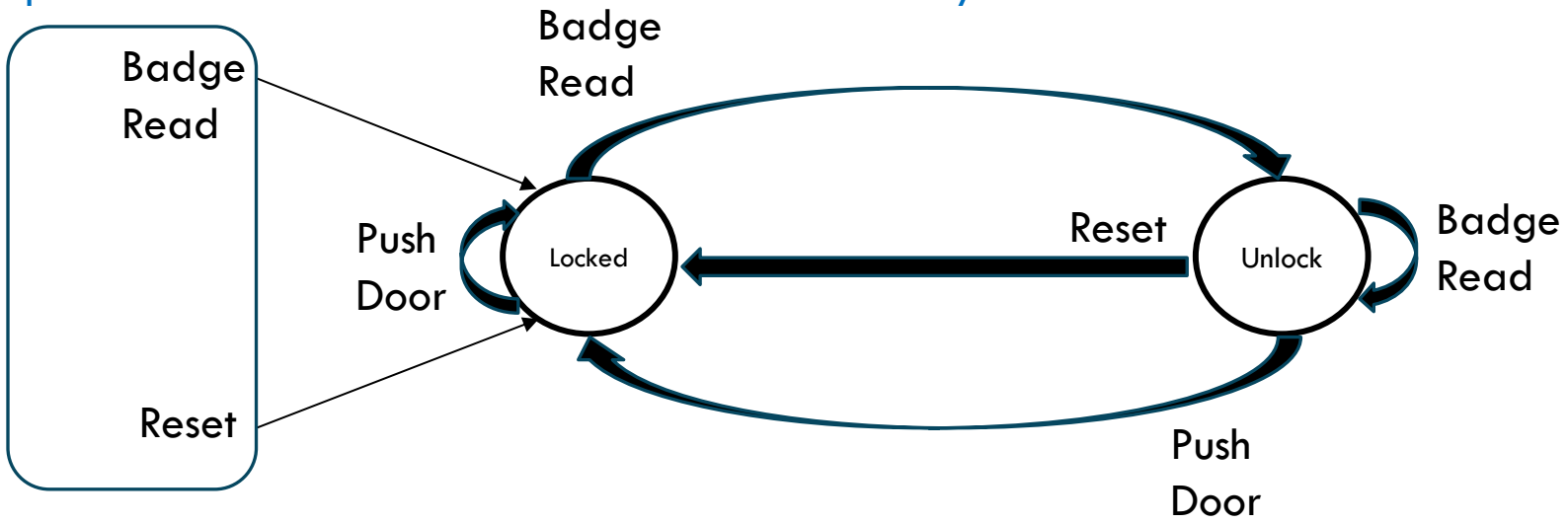
- Best practice is to use blocking assignments for combinational always blocks

Ex.

```
Always @(a or b or c)
begin
 x = a|b;
 y = a^b;
 z = a&b;
end
```

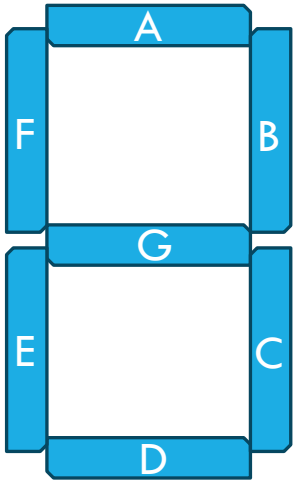
PROGRAMMING CHALLENGE 4

- Create a program that implements a state machine for the Badge Door Opener
  - state machine will have two states
    localparam LOCKED = 1'b0; //state 0
    localparam UNLOCKED = 1'b1; //state 1
  - Use always block and case statements to create FSM
  - Use the diagram below to define your inputs and outputs
  - Should be sensitive to both clock and reset (FSM not smart enough to start at state 0 so a reset when hardware is powered up assures that FSM is not in bad state)
  - Simulate using "assert" function to help verify/troubleshoot AND substitute inputs for switches that represent action taken and load on FPGA and verify on board



SEVEN SEGMENT DISPLAY

- Can be lit in various combinations to produce different patterns



4-bit	a	b	c	d	e	f	g
0000	1	1	1	1	1	1	0
0001							
0010							
0011							
0100							
0101							
0110							

7-Segment Display Truth Table Decoder

SEVEN SEGMENT DISPLAY

- Need to update the physical constraint file to include the LCD

```
# ################################

# iCEcube PCF

# Family & Device:    iCE40HX1K

# Package:            VQ100

# ################################

### Main FPGA Clock
set_io i_Clk 15

### LED Pins:
set_io o_LED_1 56
set_io o_LED_2 57
set_io o_LED_3 59
set_io o_LED_4 60

## Push-Button Switches
set_io i_Switch_1 53
set_io i_Switch_2 51
set_io i_Switch_3 54
set_io i_Switch_4 52

### 7 Segment Outputs
set_io o_Segment1_A 3
set_io o_Segment1_B 4
set_io o_Segment1_C 93
set_io o_Segment1_D 91
set_io o_Segment1_E 90
set_io o_Segment1_F 1
set_io o_Segment1_G 2
set_io o_Segment2_A 100
set_io o_Segment2_B 99
set_io o_Segment2_C 97
set_io o_Segment2_D 95
set_io o_Segment2_E 94
set_io o_Segment2_F 8
set_io o_Segment2_G 96
```

PROGRAMMING CHALLENGE 5

- Create a program for the 7 segment LCD
  - Use 7 segment display truth table to create an encoder
  - update .pcf in include LCD connections and add to module outputs
  - Use a 4-bit input to select number
  - Assign hex values for numbers 10-15 (A-F)
- Simulate to verify functionality OR make copy of code and substitute 4-bit input with 4 switches and test code on FPGA

```
module Seven_Segment_Display

(input          i_Clk,
 input [3:0]    i_Selector,
 output         o_Segment1_A,
.
.
.
.
.

 output         o_Segment1_G);

 reg [6:0] r_Name;

 always @(posedge i_Clk)
 begin
        case (i_Selector)
                4'b0000: r_Name <= 7'b1111110;
                .
                .
                .
                .
                .
                .
                .
                .

                default: r_Name <= 7'b0000000;
        endcase
 end

assign o_Segment1_A = r_Name[6];
.
.
.
.

assign o_Segment1_G = r_Name[0];
endmodule
```